

Dynamic Programming (DP) Coding Patterns

What is Dynamic Programming?

Dynamic Programming (DP) is a technique for solving problems by breaking them down into smaller subproblems, solving each once, and storing their solutions to avoid redundant computation.

Use DP When:

- The problem has overlapping subproblems.
- The problem has an optimal substructure.

DP Approaches:

1. Top-Down (Recursion + Memoization)
2. Bottom-Up (Tabulation)

Common DP Patterns & Problems:

1. Fibonacci Numbers - nth term using DP
2. 0/1 Knapsack - Maximize value within weight limit
3. Longest Common Subsequence (LCS) - Match sequences
4. Coin Change - Fewest coins to reach target amount
5. Unique Paths - Grid-based movement/counting
6. Subset Sum - Partition into equal sum subsets

Example: Bottom-Up Fibonacci

```
int fib(int n) {  
    vector<int> dp(n+1, 0);  
    dp[1] = 1;  
    for (int i = 2; i <= n; ++i)  
        dp[i] = dp[i-1] + dp[i-2];  
    return dp[n];  
}
```

Tips:

- Identify repeating subproblems.
- Use arrays or hash maps to store subproblem results.

- Try recursive solution first, then optimize with DP.